Manuel de déploiement : Docker et Nginx, pas à pas

Ce manuel a deux objectifs :

1. **Déployer** SUM (Suivi de l'Utilisation de mSupply, anciennement « tableau de bord DQA ») en production.
2. **T'apprendre à écrire toi-même** toute la configuration Docker et Nginx, de façon à la comprendre et à la modifier sans dépendre de personne.

Il se lit dans l'ordre. Chaque section explique un concept, puis montre comment il est appliqué dans les fichiers du dépôt. Les fichiers concernés :

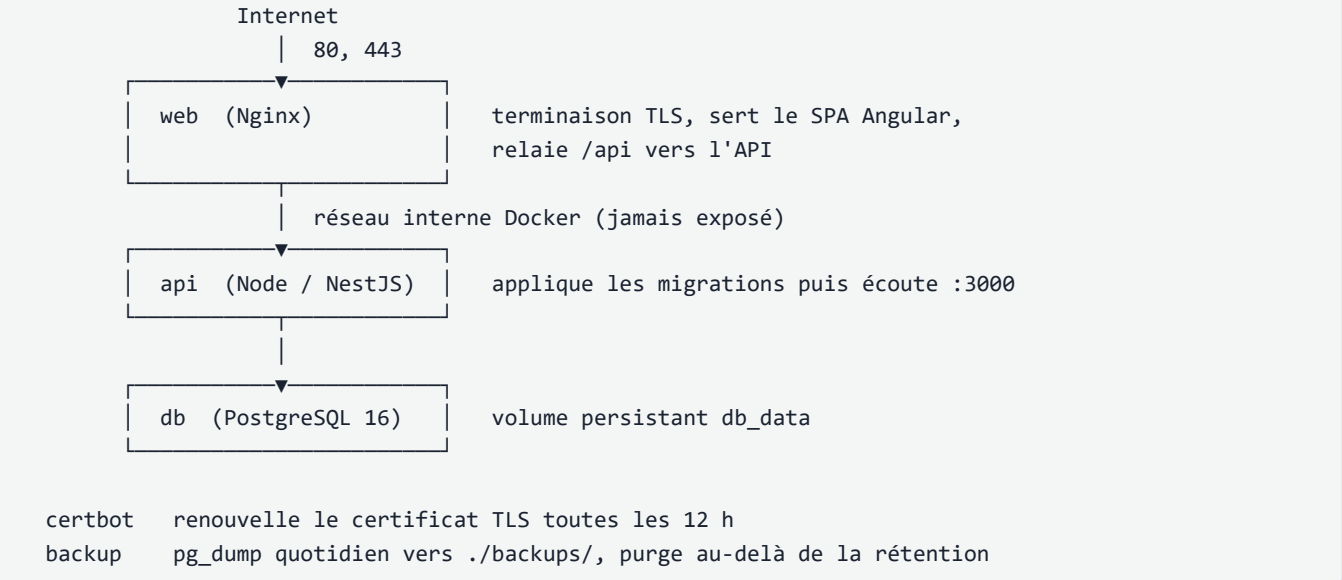
```
docker-compose.prod.yml            la pile de production
apps/api/Dockerfile                image de l'API
apps/web/Dockerfile                image du front
.dockerignore                      ce qui n'entre pas dans le build
deploy/.env.prod.example           variables (domaine, secrets, fuseau)
deploy/nginx/default.conf.template gabarit Nginx (TLS, proxy, SPA)
deploy/nginx/security-headers.conf en-têtes de sécurité partagés
deploy/nginx/99-reload.sh          rechargement Nginx après renouvellement TLS
deploy/init-letsencrypt.sh         première émission du certificat
deploy/backup.sh, deploy/restore.sh sauvegarde / restauration
```

---

1. L'architecture

1.1 Ce qui tourne

Quatre conteneurs de service et un utilitaire, sur une seule machine :



1.2 Le trajet d'une requête

1. Le navigateur ouvre `https://dqa.exemple.org/`. DNS résout vers l'IP du serveur. Le paquet arrive sur le port 443.

2. **Nginx** (`web`) termine le TLS (déchiffre), regarde le chemin.
3. `/` ou une route Angular : Nginx renvoie un fichier statique, ou `index.html` si le chemin ne correspond à aucun fichier (routage SPA).
4. `/api/...` : Nginx agit en **reverse proxy**. Il ouvre une connexion HTTP (en clair, réseau interne) vers `api:3000` , transmet la requête, renvoie la réponse.
5. **L'API** (`api`) parle à **PostgreSQL** (`db`) sur le réseau interne.
6. `db` n'a **aucun port publié** : rien depuis Internet ne l'atteint. Seuls les conteneurs du même réseau Compose la voient, par le nom `db` .

1.3 Pourquoi cette forme

- **Un seul point d'entrée** (Nginx) : un seul endroit où gérer le TLS, les en-têtes de sécurité, les limites de taille, les journaux d'accès.
- **L'API et la base ne sont pas exposées** : la surface d'attaque se limite à Nginx.
- **Tout est reproductible** : `git clone` + trois commandes redonnent la même pile sur n'importe quelle machine.

---

2. Docker : les fondamentaux

2.1 Image et conteneur

- Une **image** est un modèle en lecture seule : un système de fichiers figé plus des métadonnées (commande à lancer, port, variables). On la construit une fois.
- Un **conteneur** est une instance en cours d'exécution d'une image, avec une fine couche accessible en écriture par-dessus. On en lance autant qu'on veut à partir de la même image.

Analogie : l'image est la classe, le conteneur est l'objet.

2.2 Les couches et le cache

Une image est un empilement de **couches**. Chaque instruction d'un `Dockerfile` (`COPY` , `RUN` , ...) produit une couche. Docker **met en cache** chaque couche : si l'instruction et ses entrées n'ont pas changé, il réutilise la couche au lieu de la refaire.

Conséquence pratique, la règle d'or : placer ce qui change rarement AVANT ce qui change souvent. Dans un projet Node :

```
COPY package.json package-lock.json ./   # change rarement
RUN npm ci                               # coûteux : ~1 min. Mis en cache.
COPY . .                                 # change à chaque commit
RUN npm run build                        # rapide
```

Si on faisait `COPY . .` puis `RUN npm ci` , le moindre changement de code invaliderait le cache de `npm ci` et le rejouerait à chaque build.

2.3 Build multi-étapes

On veut **compiler** avec tout l'outillage (TypeScript, npm, sources), mais **livrer** une image minuscule sans cet outillage. Le multi-étapes sépare les deux :

```
FROM node:24-slim AS build      # étape 1 : compile
...
RUN npm run build

FROM node:24-slim AS runtime    # étape 2 : image finale
COPY --from=build /app/dist ./dist   # ne récupère QUE le résultat
```

L'image finale ne contient pas les sources, ni le cache npm de dev, ni le compilateur. Elle est plus petite et a moins de failles potentielles.

2.4 `.dockerignore`

Quand on lance `docker build .`, Docker envoie **tout le dossier** (le « contexte ») au moteur. `node_modules/`, `.git/`, les dumps SQL : inutiles et lourds. `.dockerignore` les exclut, comme `.gitignore` pour git.

Le nôtre (`/.dockerignore`) exclut `node_modules`, `dist`, `.git`, `.env*`, les `.xlsx`, etc. `!.env.example` ré-inclut l'exemple.

---

3. Le Dockerfile, ligne par ligne

3.1 `apps/api/Dockerfile`

```
# syntax=docker/dockerfile:1
```

Active la syntaxe Dockerfile moderne (peu importe la version de Docker installée).

```
FROM node:24-slim AS build
WORKDIR /app
```

`FROM` : image de départ. `node:24-slim` = Node 24 sur une base Debian réduite. `AS build` nomme l'étape. `WORKDIR` : dossier de travail (créé si absent), toutes les instructions suivantes s'y exécutent.

```
COPY package.json package-lock.json ./
COPY packages/contracts/package.json packages/contracts/
COPY apps/api/package.json apps/api/
RUN npm ci
```

On copie **seulement les manifestes** d'abord, puis on installe. `npm ci` installe exactement ce que décrit `package-lock.json` (reproductible). Tant que les manifestes ne changent pas, cette couche coûteuse reste en cache (cf. 2.2).

```
COPY tsconfig.base.json ./
COPY packages/contracts packages/contracts
COPY apps/api apps/api
COPY db/migrations db/migrations
RUN npm run build -w @dqa/contracts \
 && npm run build -w @dqa/api \
 && npm prune --omit=dev
```

Maintenant on copie les sources et on compile. `-w` cible un workspace du monorepo. `npm prune --omit=dev` retire les dépendances de développement de `node_modules` : on garde un `node_modules` prêt pour la production.

```
FROM node:24-slim AS runtime
ENV NODE_ENV=
WORKDIR /app
```

Deuxième étape, image finale. `ENV NODE_ENV=` la laisse vide **dans l'image** ; c'est le `docker-compose.prod.yml` qui la fixera à `production` (voir 4.5). Pourquoi vide par défaut : la démo tourne en HTTP et le cookie de session ne doit alors pas être `Secure`.

```
COPY --from=build /app/node_modules node_modules
COPY --from=build /app/apps/api/dist apps/api/dist
COPY --from=build /app/db/migrations db/migrations
...
COPY apps/api/docker-entrypoint.sh /usr/local/bin/entrypoint
RUN chmod +x /usr/local/bin/entrypoint
```

On récupère de l'étape `build` : les dépendances élaguées, le JavaScript compilé, les migrations SQL. Plus le script de démarrage.

```
USER node
```

L'image `node` fournit déjà un utilisateur non privilégié `node` (uid 1000). On bascule dessus : si un jour l'API est compromise, l'attaquant n'est pas `root` dans le conteneur. Notre runtime n'écrit rien sur le disque, donc aucun souci de permissions.

```
EXPOSE 3000
```

Documentaire uniquement. N'ouvre aucun port. Indique à qui lit l'image quel port l'application écoute.

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \
  CMD node -e "fetch('http://localhost:'+(process.env.PORT||3000)+'/api/sante')..."
```

Docker exécute cette commande périodiquement dans le conteneur. Si elle échoue `retries` fois de suite, le conteneur passe `unhealthy` ; avec `restart: unless-stopped` et les `depends_on: condition: service_healthy`, la pile réagit. `start-period` = délai de grâce au démarrage (le temps que les migrations passent).

```
ENTRYPOINT ["sh", "/usr/local/bin/entrypoint"]
```

Commande lancée au démarrage du conteneur. Le script (`apps/api/docker-entrypoint.sh`) fait :

```
node dist/db/migrate.js up   # applique les migrations (idempotent)
exec node dist/main.js       # puis démarre l'API. `exec` = remplace le shell,
                             # l'API devient PID 1 et reçoit les signaux d'arrêt.
```

`ENTRYPOINT` vs `CMD` : `ENTRYPOINT` est la commande, `CMD` en fournit les arguments par défaut (surchargeables sur la ligne `docker run`). Ici on n'a que `ENTRYPOINT`.

3.2 `apps/web/Dockerfile`

Même schéma en deux étapes. L'étape `build` compile le SPA Angular (`npm run build -w @dqa/web`) ; Angular émet les fichiers dans `apps/web/dist/browser/`.

```
FROM nginx:1.27-alpine AS runtime
COPY apps/web/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/apps/web/dist/browser /usr/share/nginx/html
EXPOSE 8080
```

L'étape finale part de l'image **Nginx**, y dépose une config (celle de la démo) et les fichiers statiques. En production, on **remonte par-dessus** une autre config (voir 4.6 et 5) : l'image reste la même, seul le fichier de config diffère.

---

4. Docker Compose

`docker-compose.prod.yml` décrit **plusieurs conteneurs, leur réseau, leurs volumes et leur ordre de démarrage** dans un seul fichier YAML. On l'exécute avec :

```
docker compose --env-file deploy/.env.prod -f docker-compose.prod.yml up -d
```

- `--env-file` : d'où viennent les variables `${...}` du fichier.
- `-f` : quel fichier compose (par défaut `docker-compose.yml`).
- `up -d` : construit si besoin, crée le réseau, lance tout en arrière-plan (`-d`).

4.1 Un service

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}']
      interval: 10s
      timeout: 5s
      retries: 12
    restart: unless-stopped
```

- `image` : l'image à utiliser telle quelle (pas de build).
- `environment` : variables passées au conteneur. `${POSTGRES_USER}` est remplacé par Compose, à partir du `--env-file`.
- `volumes` : ici, le **volume nommé** `db_data` monté sur le dossier de données de PostgreSQL. Sans lui, un `docker compose down` effacerait la base.
- `healthcheck` : ici on surcharge celui de l'image. `pg_isready` répond quand PostgreSQL accepte les connexions.
- `restart: unless-stopped` : Docker relance le conteneur s'il plante, ou au redémarrage de la machine, sauf si on l'a arrêté à la main.

4.2 Un service construit sur place

```
api:
  image: dqa-api:latest
  build:
    context: .
    dockerfile: apps/api/Dockerfile
```

`build.context` = le dossier envoyé au moteur (racine du monorepo, pour que le Dockerfile voie `packages/contracts` et `apps/api`). `dockerfile` = quel Dockerfile dans ce contexte. `image` nomme le résultat.

4.3 Le réseau

Compose crée **automatiquement un réseau** pour la pile. Chaque service y est joignable **par son nom** : l'API se connecte à la base via l'hôte `db`, Nginx proxyfie vers `api`. C'est un DNS interne (résolveur à `127.0.0.11`).

Seuls les ports explicitement `ports:` sont publiés vers l'hôte. Ici, seul `web` publie `80` et `443`. `api` et `db` ne sont accessibles que depuis le réseau interne.

```
web:
  ports:
    - '80:80'      #  hôte:conteneur
    - '443:443'
```

4.4 Volumes : nommés contre bind mounts

- **Volume nommé** (`db_data:/var/lib/postgresql/data`) : Docker gère l'emplacement sur disque. Pour des **données** (base, certificats). Survit à `down`, disparaît avec `down -v`.
- **Bind mount** (`./deploy/nginx/default.conf.template:/etc/nginx/templates/...`) : un fichier/dossier **de l'hôte** monté dans le conteneur. Pour de la **configuration** qu'on veut éditer sans reconstruire l'image. `:ro` = lecture seule.

Notre `web` combine les deux : bind mounts pour les configs Nginx, volumes nommés pour les certificats partagés avec `certbot`.

4.5 Variables et `--env-file`

`deploy/.env.prod` (copié de `.env.prod.example`, **jamais committé**) contient :

```
DQA_DOMAIN=dqa.exemple.org
POSTGRES_PASSWORD=...
DQA_TZ=Africa/Abidjan
```

Dans le compose, `${DQA_DOMAIN}` y est remplacé. On peut composer des valeurs :

```
DATABASE_URL: postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
```

Piège du `$` : dans une commande shell écrite dans le compose, un `$` simple serait interprété par Compose comme une variable. Pour un `$` **littéral** (destiné au shell du conteneur), on écrit `$$` :

```
entrypoint: >
  sh -c 'ts=$$(date +%Y%m%d); pg_dump ... /backups/dqa-$$ts.dump'
```

`$$(date)` devient `$(date)` dans le conteneur.

4.6 `depends_on` et l'ordre de démarrage

```
  api:
    depends_on:
      db:
        condition: service_healthy
```

`api` ne démarre que quand `db` est `healthy` (pas juste « lancé »). De même `web` attend `api`. Cela évite les erreurs « connection refused » au boot.

4.7 Nos cinq services

Service	Rôle	Ports	Volume
`db`	PostgreSQL 16	aucun	`db_data` (persistant)
`api`	NestJS, migrations + API	aucun	aucun
`web`	Nginx : TLS, SPA, proxy `/api`	80, 443	configs (bind), certs (nommés)
`certbot`	renouvellement TLS toutes les 12 h	aucun	`certbot_certs`, `certbot_webroot`
`backup`	`pg_dump` quotidien + purge	aucun	`./backups` (bind)

---

5. Nginx

5.1 Le modèle de configuration

Un fichier Nginx est une hiérarchie de **contextes** contenant des **directives** :

```
# contexte "main" (implicite, tout le fichier)
resolver 127.0.0.11;                # une directive

server {                            # contexte "server" : un site virtuel
    listen 443 ssl;
    server_name dqa.exemple.org;

    location /api/ {                 # contexte "location" : un préfixe d'URL
        proxy_pass http://api:3000;
    }
}
```

- Une **directive** finit par `;`. Un **bloc** est délimité par `{ }`.
- `server` = un hôte virtuel, choisi selon `listen` (port) + `server_name` (nom d'hôte de la requête).
- `location` = un ensemble de règles pour un sous-ensemble d'URL. Nginx choisit **une seule** location par requête (la plus spécifique).
- **Héritage :** une directive posée dans `server` s'applique dans ses `location` … **sauf** `add_header` (voir 5.6, le piège classique).

Notre gabarit `deploy/nginx/default.conf.template` a **deux** `server` : un sur le port 80, un sur le port 443.

5.2 Servir un SPA (single page application)

Angular gère le routage côté navigateur. Le serveur doit renvoyer `index.html` pour toute URL qui ne correspond pas à un fichier réel, sinon un rafraîchissement sur `/admin/strates` donnerait un 404.

```
location / {
    try_files $uri $uri/ /index.html;
}
```

`try_files` essaie dans l'ordre : le fichier `$uri`, puis le dossier `$uri/`, puis, en dernier recours, `/index.html`. `$uri` est la variable Nginx du chemin demandé.

5.3 Le cache des fichiers

Angular ajoute une empreinte au nom des fichiers compilés (`main-8F3A1C.js`). Le contenu d'un nom donné ne change **jamais** : on peut le mettre en cache pour un an.

```
location ~* \.(?:js|css|woff2?|png|svg|ico|webp)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    access_log off;
}
```

`~*` = correspondance par expression régulière, insensible à la casse. `index.html`, lui, garde `Cache-Control: no-cache` : le navigateur revérifie à chaque visite, sinon un déploiement passerait inaperçu.

5.4 Le reverse proxy

```
location /api/ {
    set $upstream http://api:3000;
    proxy_pass $upstream;
    proxy_http_version 1.1;
    proxy_set_header Host              $host;
    proxy_set_header X-Real-IP         $remote_addr;
    proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 300s;
}
```

- `proxy_pass` : transmet la requête à `http://api:3000`. Le chemin `/api/foo` est transmis tel quel (l'API attend le préfixe `/api`).
- **Pourquoi `set $upstream` puis `proxy_pass $upstream` ?** Avec une adresse **littérale**, Nginx résout `api` **une fois au démarrage** et fige l'IP. Si le conteneur `api` redémarre avec une autre IP, Nginx envoie dans le vide. Avec une **variable**, Nginx re-résout à chaque requête, via le `resolver` déclaré en tête (le DNS interne de Docker, `127.0.0.11`).
- `proxy_set_header` : Nginx réécrit certains en-têtes. `Host` = le nom demandé par le client ; `X-Forwarded-For` / `X-Forwarded-Proto` disent à l'API « la requête d'origine venait de telle IP, en HTTPS » (utile pour les logs et les cookies `Secure`).
- `proxy_read_timeout 300s` : un import de classeur déclenche le gel et la reconstruction des vues matérialisées ; ça peut durer une à deux minutes.

5.5 TLS (HTTPS)

```
server {
    listen 443 ssl;
    http2 on;
    server_name dqa.exemple.org;

    ssl_certificate         /etc/letsencrypt/live/dqa.exemple.org/fullchain.pem;
    ssl_certificate_key     /etc/letsencrypt/live/dqa.exemple.org/privkey.pem;
    ssl_trusted_certificate /etc/letsencrypt/live/dqa.exemple.org/chain.pem;

    ssl_protocols             TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers off;
    ssl_ciphers               ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:...;
    ssl_session_cache         shared:TLS:10m;
    ssl_session_timeout       1d;
    ssl_session_tickets       off;

    ssl_stapling        on;
    ssl_stapling_verify on;
}
```

- `listen 443 ssl` : ce `server` termine le TLS. `http2 on` active HTTP/2 (multiplexage des requêtes sur une connexion).
- `ssl_certificate` = chaîne complète (certificat du domaine + intermédiaires) ; `ssl_certificate_key` = clé privée (jamais exposée) ; `ssl_trusted_certificate` = la chaîne, pour l'agrafage OCSP.
- `ssl_protocols TLSv1.2 TLSv1.3` : on refuse TLS 1.0/1.1 (obsolètes).

- `ssl_ciphers` : liste des suites autorisées, toutes en **ECDHE** (secret persistant : compromettre la clé plus tard ne déchiffre pas le trafic passé) et **AEAD** (GCM / ChaCha20). `ssl_prefer_server_ciphers off` : en TLS 1.3, le client choisit, c'est très bien.
- `ssl_session_cache` / `session_timeout` : reprise de session, évite une poignée de main complète à chaque requête. `session_tickets off` : les tickets affaiblissent le secret persistant si la clé de ticket fuit.
- `ssl_stapling` : Nginx récupère lui-même la preuve « ce certificat n'est pas révoqué » et la joint à la poignée de main. Le navigateur n'a pas à contacter l'autorité (plus rapide, plus privé). Nécessite un `resolver` (déjà présent).

5.6 En-têtes de sécurité, et LE piège `add_header`

```
add_header Strict-Transport-Security "max-age=63072000" always;
add_header X-Content-Type-Options    "nosniff"          always;
add_header X-Frame-Options           "SAMEORIGIN"       always;
add_header Referrer-Policy           "strict-origin-when-cross-origin" always;
```

- **HSTS** (`Strict-Transport-Security`) : le navigateur, une fois vu cet en-tête, refuse tout HTTP en clair vers ce domaine pendant `max-age` secondes (ici 2 ans). À n'activer **qu'une fois sûr** que le HTTPS marche : on ne peut plus revenir en arrière côté navigateurs des visiteurs.
- `X-Content-Type-Options: nosniff` : le navigateur ne « devine » pas le type d'un fichier, il fait confiance au `Content-Type` .
- `X-Frame-Options: SAMEORIGIN` : la page ne peut être mise en `<iframe>` que par le même domaine (anti-clickjacking).
- `Referrer-Policy` : limite ce qui est envoyé dans l'en-tête `Referer` .
- `always` : ajoute l'en-tête **même sur les réponses d'erreur** (301, 404, 500). Sans `always` , seules les 2xx/3xx le reçoivent.

Le piège : `add_header` **ne s'hérite pas** comme les autres directives. Si un bloc `location` contient **ne serait-ce qu'un** `add_header` , il **perd tous** les `add_header` du `server` parent.

Notre `location /` pose `Cache-Control: no-cache` . Elle perdrait donc HSTS et compagnie. La solution : un fichier partagé, ré-inclus partout où c'est nécessaire :

```
# deploy/nginx/security-headers.conf : les 4 add_header ci-dessus

server {
    include /etc/nginx/snippets/security-headers.conf;   # pour /api/ et les erreurs

    location / {
        include /etc/nginx/snippets/security-headers.conf;   # re-inclus ici
        add_header Cache-Control "no-cache";
    }
    location ~* \.(?:js|css|...)$ {
        include /etc/nginx/snippets/security-headers.conf;   # et ici
        add_header Cache-Control "public, immutable";
    }
}
```

`include` insère littéralement le contenu d'un autre fichier. C'est le mécanisme pour factoriser.

5.7 La compression

```
gzip            on;
gzip_vary       on;
gzip_min_length 1024;
gzip_types      text/plain text/css application/json application/javascript image/svg+xml;
```

Nginx compresse à la volée les réponses textuelles de plus de 1 Ko. `gzip_vary` ajoute `Vary: Accept-Encoding` (les caches intermédiaires servent la bonne version). On ne compresse pas les `.woff2` / images : déjà compressés, ce serait du CPU pour rien.

5.8 Redirection HTTP vers HTTPS et challenge ACME

Le `server` du port 80 fait deux choses :

```
server {
    listen 80;
    server_name dqa.exemple.org;

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;      # Let's Encrypt viendra lire un jeton ici
    }
    location / {
        return 301 https://$host$request_uri;   # tout le reste -> HTTPS
    }
}
```

`return 301` = redirection permanente. `$host` = le nom demandé, `$request_uri` = chemin + paramètres. On **garde le port 80 ouvert** uniquement pour le renouvellement du certificat (section 6).

5.9 `envsubst` : le gabarit

Le fichier s'appelle `default.conf.template`, pas `.conf`. L'image Nginx, à son démarrage, exécute `envsubst` sur tous les `*.template` de `/etc/nginx/templates/` et écrit le résultat dans `/etc/nginx/conf.d/`.

`envsubst` remplace **uniquement** les `${VARIABLE}` présentes dans l'environnement du conteneur. On passe `DQA_DOMAIN` en `environment:`, donc `${DQA_DOMAIN}` est substitué. Les variables **Nginx** (`$host`, `$uri`, ...) n'ont pas d'accolades et ne sont pas dans l'environnement : elles sont laissées telles quelles. **Règle : `${...}` = substitué au démarrage ; `$...` = variable Nginx à l'exécution.**

Comme le fichier produit est `/etc/nginx/conf.d/default.conf`, il **remplace** la config démo bakée dans l'image.

5.10 Le hook de rechargement

`deploy/nginx/99-reload.sh`, monté dans `/docker-entrypoint.d/`, est lancé par l'entrypoint de l'image Nginx **avant** le démarrage du serveur :

```
( while :; do sleep 6h; nginx -s reload 2>/dev/null || true; done ) &
```

Il passe une boucle en tâche de fond : toutes les 6 h, `nginx -s reload` recharge la config (donc relit les fichiers de certificat) sans couper le service. C'est ainsi qu'un certificat renouvelé par `certbot` est pris en compte.

> Note : l'image Nginx n'exécute les scripts de `/docker-entrypoint.d/` (dont `envsubst`) **que si** sa commande commence par `nginx`. C'est pourquoi on ne surcharge PAS `command:` dans le compose, et qu'on passe par ce hook.

---

6. Let's Encrypt et certbot

6.1 Le principe (ACME, challenge HTTP-01)

Let's Encrypt délivre gratuitement des certificats valables 90 jours, via le protocole **ACME**. Pour prouver qu'on contrôle `dqa.exemple.org`, le **challenge HTTP-01** :

1. certbot demande un certificat pour le domaine.
2. Let's Encrypt répond « place le jeton `XYZ` à `http://dqa.exemple.org/.well-known/acme-challenge/XYZ` ».
3. certbot écrit le jeton dans un dossier partagé ; Nginx le sert (port 80).
4. Let's Encrypt va le lire. Si ça correspond : certificat émis.

D'où : **le port 80 doit rester ouvert et joignable depuis Internet**, et le DNS du domaine doit pointer sur le serveur.

6.2 Les volumes partagés

```
  web:
    volumes:
      - certbot_certs:/etc/letsencrypt:ro       # lit les certificats
      - certbot_webroot:/var/www/certbot:ro     # sert les jetons
  certbot:
    volumes:
      - certbot_certs:/etc/letsencrypt           # écrit les certificats
      - certbot_webroot:/var/www/certbot         # écrit les jetons
```

Deux volumes nommés, partagés entre `web` (lecture seule) et `certbot` (lecture-écriture).

6.3 Le problème de l'amorçage (et sa solution)

Nginx **refuse de démarrer** si un fichier `ssl_certificate` référencé n'existe pas. Mais on ne peut obtenir le certificat qu'avec Nginx démarré (pour servir le challenge). Impasse.

`deploy/init-letsencrypt.sh` la résout, en 5 étapes :

1. Génère un **certificat provisoire auto-signé** (`openssl req -x509`) à l'emplacement attendu. Nginx peut démarrer.
2. Démarre `web`.
3. Supprime le provisoire.
4. Lance `certbot certonly --webroot` : le vrai challenge, le vrai certificat.
5. `nginx -s reload` (recharge le vrai certificat), puis `up -d` (toute la pile).

À lancer **une seule fois**. Ensuite le service `certbot` tourne en boucle :

```
  certbot:
    entrypoint: sh -c "trap exit TERM; while :; do certbot renew --webroot -w /var/www/certbot --quiet;
```

`certbot renew` ne fait rien tant que le certificat n'est pas dans sa fenêtre de renouvellement (30 jours avant expiration) ; sinon il le renouvelle. Le hook Nginx (5.10) le recharge dans les 6 h.

6.4 Mode staging

`DQA_ACME_STAGING=1` dans `.env.prod` fait passer par le serveur de **test** de Let's Encrypt : quota très large, mais certificat **non reconnu** par les navigateurs. Utile pour répéter la procédure sans épuiser le quota réel (5 échecs / heure, 50 certificats / semaine par domaine). Repasser à `0` pour de vrai.

---

7. Mise en service, pas à pas

Sur le serveur (Debian/Ubuntu à jour, Docker installé) :

```
# 1. Récupérer le code
git clone https://github.com/yalatif0210/msupply_usage_tracking.git
cd msupply_usage_tracking

# 2. DNS : créer un enregistrement A (et AAAA si IPv6) qui pointe
#    dqa.exemple.org -> IP publique du serveur. Vérifier :
dig +short dqa.exemple.org

# 3. Pare-feu : ouvrir 80 et 443 entrants
sudo ufw allow 80,443/tcp        # (ou groupe de sécurité du fournisseur)

# 4. Configuration
cp deploy/.env.prod.example deploy/.env.prod
$EDITOR deploy/.env.prod
#   DQA_DOMAIN=dqa.exemple.org
#   DQA_ACME_EMAIL=exploitation@exemple.org
#   POSTGRES_PASSWORD=$(openssl rand -base64 24)
#   DQA_TZ=Africa/Abidjan

# 5. Construire les images
docker compose --env-file deploy/.env.prod -f docker-compose.prod.yml build

# 6. Émettre le certificat TLS (une fois). Conseil : d'abord DQA_ACME_STAGING=1
#    pour un essai, puis remettre à 0 et relancer.
chmod +x deploy/*.sh
./deploy/init-letsencrypt.sh

# 7. Démarrer toute la pile
docker compose --env-file deploy/.env.prod -f docker-compose.prod.yml up -d

# 8. Vérifier
docker compose --env-file deploy/.env.prod -f docker-compose.prod.yml ps
curl -I https://dqa.exemple.org/
docker compose --env-file deploy/.env.prod -f docker-compose.prod.yml logs -f api
```

Première connexion : `coordination` / le mot de passe de seed (`coordination-2026`). **Le changer immédiatement** (écran Comptes, rôle `admin`), et changer aussi `admin` et `lecteur` . Les comptes de seed sont créés par la migration `0010` ; ses mots de passe sont documentés comme « à changer avant mise en service ».

7.1 Astuce : un alias

```
echo "alias dc='docker compose --env-file deploy/.env.prod -f docker-compose.prod.yml'" >> ~/.bashrc
source ~/.bashrc
# désormais :  dc ps   |   dc logs -f api   |   dc up -d
```

---

8. Exploitation

8.1 Journaux

```
dc logs -f            # tout, en continu
dc logs --tail=100 api
dc logs web | grep -v ' 200 '     # requêtes non-200
```

8.2 Déployer une nouvelle version

```
git pull
dc build
dc up -d           # recrée seulement les conteneurs dont l'image a changé
```

Au redémarrage, `api` rejoue `migrate up` : les nouvelles migrations SQL s'appliquent automatiquement, les anciennes sont ignorées (registre `public.schema_migrations`). Aucune action manuelle sur la base.

8.3 Sauvegardes

Le service `backup` fait un `pg_dump` **custom** (compressé, restaurable sélectivement) chaque jour dans `./backups/` , et supprime les dumps plus vieux que `DQA_BACKUP_RETENTION_DAYS` .

```
./deploy/backup.sh                                  # sauvegarde immédiate
ls -lh backups/
./deploy/restore.sh backups/dqa-20260906-030000.dump # restauration
```

Règle des 3-2-1 : au moins 3 copies, sur 2 supports, dont 1 hors site. Copier `./backups/` vers un stockage externe (rsync, S3, …) via une tâche cron de l'hôte :

```
30 3 * * *  cd /srv/msupply_usage_tracking && ./deploy/backup.sh && rsync -a backups/ ailleurs:/backups/
```

Sauvegarder aussi `deploy/.env.prod` (secrets) dans un gestionnaire de secrets, séparément.

8.4 Supervision minimale

- `dc ps` : tous les services doivent être `Up` et, pour `api` / `db` , `healthy` .

- Certificat : `dc exec web openssl x509 -enddate -noout -in /etc/letsencrypt/live/$DQA_DOMAIN/fullchain.pem`
- Espace disque : `df -h` (les volumes Docker et `./backups/` grossissent).
- Une sonde externe (UptimeRobot, healthchecks.io) sur `https://dqa.exemple.org/api/sante`.

8.5 Dépannage

Symptôme	Piste
`web` redémarre en boucle	`dc logs web` : souvent `ssl_certificate` introuvable -> relancer `init-letsencrypt.sh`
502 Bad Gateway sur `/api`	`api` non `healthy` : `dc logs api` (migrations ? `DATABASE_URL` ?)
certbot échoue	port 80 fermé, DNS pas encore propagé, ou quota (utiliser staging)
Cookie de session non conservé	`NODE_ENV` doit valoir `production` (cookie `Secure`) ET le site en HTTPS
Dates de clôture P2/P3 décalées	vérifier `DQA_TZ` (conteneurs `db` et `api`)
Build lent à chaque fois	un `COPY` de source avant `npm ci` casse le cache (cf. 2.2)

---

9. Annexes

9.1 Variables (`deploy/.env.prod`)

Variable	Rôle
`DQA_DOMAIN`	nom de domaine public, injecté dans Nginx par `envsubst`
`DQA_ACME_EMAIL`	contact Let's Encrypt (expiration, incidents)
`DQA_ACME_STAGING`	`1` = serveur de test Let's Encrypt
`DQA_TZ`	fuseau des conteneurs `db` et `api` (`Africa/Abidjan`)
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	base ; `DATABASE_URL` en est dérivé
`DQA_BACKUP_RETENTION_DAYS`	jours de rétention des dumps

9.2 Ports

Port	Qui	Exposé ?
80	Nginx : challenge ACME + redirection	oui (Internet)
443	Nginx : l'application	oui (Internet)
3000	API	non (réseau interne)
5432	PostgreSQL	non (réseau interne)

9.3 Commandes de référence

```
dc up -d                # démarrer / mettre à jour
dc down                 # arrêter (SANS toucher aux volumes -> données gardées)
dc down -v              # arrêter ET tout effacer (JAMAIS en prod)
dc ps                   # état
dc logs -f <service>    # journaux
dc restart <service>    # redémarrer un service
dc exec <service> sh    # ouvrir un shell dans un conteneur
dc build <service>      # reconstruire une image
dc pull                 # tirer les images de base à jour (postgres, nginx, certbot)
```

9.4 Checklist de mise en production

- ☐ DNS `DQA_DOMAIN` -> IP serveur (A, et AAAA si IPv6)
- ☐ Ports 80 et 443 ouverts en entrée
- ☐ `deploy/.env.prod` renseigné, `POSTGRES_PASSWORD` fort et unique
- ☐ `DQA_TZ` correct
- ☐ `./deploy/init-letsencrypt.sh` passé (essai staging puis réel)
- ☐ `curl -I https://DQA_DOMAIN/` -> `200`, `Strict-Transport-Security` présent
- ☐ Test SSL externe (ssllabs.com) -> note A
- ☐ Mots de passe des comptes `coordination`, `admin`, `lecteur` changés
- ☐ Sauvegarde testée : `backup.sh` puis `restore.sh` sur une base jetable
- ☐ Copie hors site de `./backups/` planifiée
- ☐ Sonde externe sur `/api/sante`

9.5 Variante : réseau interne sans domaine public

Si le tableau de bord n'est accessible que sur un réseau local (pas de domaine, pas d'accès Let's Encrypt) :

- Remplacer le `server` 443 par un `server` 80 servant directement l'application (retirer tout le bloc `ssl_*` et la redirection).
- Retirer les services `certbot` et le hook `99-reload.sh`.
- Mettre `NODE_ENV` à vide (et non `production`) pour l'API : le cookie de session ne doit pas être `Secure` en HTTP. Attention : le trafic (dont les mots de passe) circule alors en clair sur le LAN.
- Ou : générer un certificat auto-signé (ou issu d'une PKI interne), le monter dans `certbot_certs`, garder le `server` 443. Les postes doivent faire confiance à l'autorité interne.